useRef()

What is useRef?

useRef() returns a mutable object with a .current property.

const refObject = useRef(initialValue);

- The value inside .current can be changed.
- Updating .current does NOT re-render the component.
- It persists between renders.

Why useRef?

1. Access DOM elements directly
2. Store previous values
3. Store mutable data without re-render

Structure

```
const myRef = useRef(initialValue);

// Access value
myRef.current
```

Example 1: Access DOM Element (Focus Input)

```
import React, { useRef } from "react";

function FocusInput() {
  const inputRef = useRef(null);

  const handleFocus = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={handleFocus}>Focus Input</button>
    </div>
  );
}
```

How it works:

- inputRef is attached to <input>
- When button is clicked → .focus() runs
- Component does NOT re-render

Example 2: Store Previous Value

```
import React, { useState, useEffect, useRef } from "react";

function PreviousValue() {
  const [count, setCount] = useState(0);
  const prevCount = useRef();

  useEffect(() => {
    prevCount.current = count;
  });

  return (
    <div>
      <h2>Current: {count}</h2>
      <h3>Previous: {prevCount.current}</h3>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}
```

Why not useState for previous value?

Because updating state causes re-render.
useRef stores value silently without re-render.

---

useMemo()

What is useMemo?

useMemo() memoizes (caches) a computed value.

```
const memoValue = useMemo(() => {
   return calculation;
}, [dependencies]);
```

It only recalculates when dependencies change.

Why useMemo?

- Avoid expensive recalculations

- Improve performance
- Prevent unnecessary computation

Example 1: Expensive Calculation

```
import React, { useState, useMemo } from "react";

function ExpensiveComponent() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("");

  const expensiveCalculation = (num) => {
    console.log("Calculating...");
    for (let i = 0; i < 1000000000; i++) {}
    return num * 2;
  };

  const result = useMemo(() => {
    return expensiveCalculation(count);
  }, [count]);

  return (
    <div>
      <h2>Result: {result}</h2>
      <button onClick={() => setCount(count + 1)}>Increase</button>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Type something"
      />
    </div>
  );
}
```

Without useMemo:

- Every time you type in input → calculation runs again.

With useMemo:

- Calculation runs only when count changes.

Important Rule

Do NOT overuse useMemo.
Only use it when:

- Computation is expensive
- Performance issue exists

---

useCallback()

 What is useCallback?

useCallback() memoizes a function.

```
const memoFunction = useCallback(() => {
  // function logic
}, [dependencies]);
```

It returns the same function reference unless dependencies change.

Why useCallback?

Prevents unnecessary re-rendering of child components.

Especially useful when passing functions as props.

Example: Parent & Child Re-render Issue

```
import React, { useState, useCallback } from "react";

const Child = React.memo(({ handleClick }) => {
  console.log("Child Rendered");
  return <button onClick={handleClick}>Click Me</button>;
});

function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    console.log("Button clicked");
  }, []);

  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>Increase</button>
      <Child handleClick={handleClick} />
    </div>
  );
}
```

What is happening?

Without useCallback:

- Parent re-renders
- New function is created
- Child re-renders

With useCallback:

- Same function reference
- Child does NOT re-render unnecessarily

Feature	useMemo	useCallback
Returns	Memoized value	Memoized function
Purpose	Optimize heavy calculations	Prevent unnecessary re-renders
Usage	const value = useMemo(...)	const fn = useCallback(...)

---

useRef()

useRef is a hook that returns a mutable object with a property called `current`.

Syntax:
const ref = useRef(initialValue);

The important points are:

- The value stored in ref.current can be changed.
- Changing ref.current does NOT cause the component to re-render.
- The value persists between renders.

There are two main uses of useRef:

1. Accessing DOM elements directly.
2. Storing mutable values that should not trigger re-render.

Example – Accessing DOM:

If you want to focus an input field when a button is clicked, you attach the ref to the input element and then use ref.current.focus() inside a function. React gives you direct access to that DOM element.

Example – Storing previous value:

You can store the previous state value inside a ref inside useEffect. Since updating ref.current does not trigger re-render, it is efficient for storing temporary or previous values.

Difference between useRef and useState:

- useState causes re-render when updated.
- useRef does not cause re-render when updated.

---

useMemo()

useMemo is used to memoize (cache) a computed value.

Syntax:
```
const memoizedValue = useMemo(() => {
return calculation;
}, [dependencies]);
```

It only recalculates the value when the dependencies change.

Why useMemo is needed:

In React, when a component re-renders, everything inside it runs again. If you have a heavy or expensive calculation, it will run every time unless optimized.

useMemo prevents unnecessary recalculation by storing the result and reusing it.

Example scenario:

Suppose you have:

- A counter state
- A text input state
- A heavy calculation based on the counter

Without useMemo:
Typing in the input will re-render the component and run the heavy calculation again.

With useMemo:
The calculation runs only when the counter changes, not when the input changes.

Important note:
Do not use useMemo everywhere. It should only be used when:

- The computation is expensive
- There is a real performance issue

---

useCallback()

useCallback is used to memoize functions.

Syntax:
```
const memoizedFunction = useCallback(() => {
function logic
}, [dependencies]);
```

It returns the same function reference unless dependencies change.

Why useCallback is needed:

In JavaScript, every time a component re-renders, new function instances are created. If you pass a function as a prop to a child component, the child may re-render because the function reference changes.

useCallback keeps the same function reference between renders unless dependencies change.

Example scenario:

Parent component:

- Has a state
- Passes a function to child

Without useCallback:
Every time parent re-renders, a new function is created.
Child receives new function reference.
Child re-renders unnecessarily.

With useCallback:
Function reference stays the same.
Child does not re-render unnecessarily (especially when used with React.memo).

Difference between useMemo and useCallback:

useMemo:

- Returns a memoized value.
- Used for heavy calculations.

useCallback:

- Returns a memoized function.
- Used to prevent unnecessary re-renders when passing functions as props.

Simple Summary:

useRef:
Stores mutable values or DOM references without causing re-render.

useMemo:
Caches a computed value to avoid expensive recalculations.

useCallback:
Caches a function to avoid unnecessary re-renders.